Mining Massive Data Sets – End-Term Project: Hierarchical Clustering, Linear Regression with CUR Decomposition, and Collaborative Filtering via LSH Using Apache PySpark

Pham Quoc Hung, Dinh Bui Khanh Huy, Nguyen Nam Phong, Huynh Huu Minh, Anh Hoang
Faculty of Information Technology, Ton Duc Thang University
Ho Chi Minh City, Vietnam
504048 – Mining Massive Data Sets tg_anhhoang@tdtu.edu.vn

***Abstract*—This paper presents the end-term project for *Mining Massive Data Sets* (504048) at Ton Duc Thang University. The project comprises three interconnected tasks, each targeting a classical large-scale data mining problem implemented with Apache PySpark for distributed computation.**

Task 1 implements bottom-up agglomerative hierarchical clustering on a corpus of 10,000 randomly generated strings using the Jaccard distance over 4-shingle tokenisations. Cluster representation and inter-cluster distance follow *Approach 2* for non-Euclidean spaces (most-central point as centroid). An automatic termination criterion based on a factor-γ jump in the maximum cluster diameter determines the final partition. Three-dimensional t-SNE visualisation confirms the geometric coherence of discovered clusters.

Task 2 applies PySpark `LinearRegression` to predict daily Vietnamese gold (SJC) prices using a 15-day lag window as input features. A `CURReducer` class exploits PySpark `RowMatrix` SVD to select the most informative feature columns via leverage scoring, reducing dimensionality from 15 to 5 in unit steps across 11 controlled experiments. Reducing features by 67% raises test RMSE by only 0.25 million VND/tael, demonstrating efficient information preservation.

Task 3 constructs a User–User Collaborative Filtering recommender system augmented by a Locality-Sensitive Hashing index for O(1) average-case candidate retrieval. Mean-centred cosine similarity neutralises the effect of unobserved (zero) ratings. LSH achieves a 12–14$\times$ speed-up over brute-force with less than 2% RMSE degradation across four neighbourhood sizes ($N \in \{5, 10, 20, 30\}$). Scalability experiments on an extended 20,000-rating dataset confirm that LSH latency remains approximately constant as the user population grows.

All implementations adhere to object-oriented design principles and are validated through quantitative experiments and visualisations in the accompanying Jupyter notebooks.

Completion: Task 1 – 100%; Task 2 – 100%; Task 3 – 100%; Report – 100%.

***Index Terms*—agglomerative clustering, Jaccard distance, 4-shingles, CUR decomposition, PySpark RowMatrix, linear regression, gold price prediction, collaborative filtering, locality-sensitive hashing, cosine similarity, distributed computation**

I. Introduction

Modern data-intensive applications routinely demand algorithms that scale to millions of records without exhausting memory or compute time. *Mining of Massive Datasets* (MMDS) [1] addresses this demand by providing algorithmic blueprints – shingling, random projections, matrix decompositions – that preserve statistical guarantees at scale.

This project puts three such blueprints into practice using Apache PySpark [3], an industry-standard distributed computing framework:

- **Section II** presents agglomerative hierarchical clustering in a non-Euclidean (Jaccard) space with a data-driven stopping criterion.
- **Section III** combines CUR matrix decomposition [4] with PySpark linear regression to forecast Vietnamese gold prices from historical lag features.
- **Section IV** integrates random-hyperplane LSH [5] into a user–user collaborative filtering pipeline [6], achieving sublinear candidate retrieval.

Each task follows OOP principles (dedicated classes with clean interfaces), uses PySpark RDDs or DataFrames where applicable, and is evaluated through reproducible experiments in self-contained Jupyter notebooks.

II. Task 1: Hierarchical Clustering in Non-Euclidean Spaces

A. Problem Statement

Standard agglomerative clustering assumes Euclidean geometry (centroids are coordinate-wise means). Strings have no natural vector embedding; instead their Jaccard distance over k-shingle sets provides a principled dissimilarity measure that is independent of string length and robust to local edits. *Approach 2* of hierarchical clustering in non-Euclidean spaces [1] addresses this by representing each cluster with its *most-central element* and defining inter-cluster distance as the distance between representatives.

B. Data Preparation

We generate 10,000 lowercase alphabetical strings with lengths drawn uniformly at random from $[32, 64]$. Each string s is tokenised into a set of 4-shingles (character-level 4-grams):

$$\mathcal{S}(s) = \{ s[i{:}i{+}4] \mid 0 \le i \le |s| - 4 \}. \tag{1}$$

Pairwise dissimilarity between two strings A and B is measured by the Jaccard distance [2]:

$$d_J(A, B) = 1 - J(A, B) = 1 - \frac{|\mathcal{S}(A) \cap \mathcal{S}(B)|}{|\mathcal{S}(A) \cup \mathcal{S}(B)|}. \tag{2}$$

Note $d_J \in [0, 1]$: identical strings yield $d_J = 0$; fully disjoint shingle sets yield $d_J = 1$. The corpus is persisted to `dataset.csv` with columns `index`, `string`, `shingles`. Table I summarises the dataset.

TABLE I
DATASET STATISTICS – TASK 1

Property	Value
Total strings generated	10,000
String length range	$[32, 64]$ characters
Shingle size (k)	4
Avg. shingle-set size	≈ 42
Subset used for clustering	800
Distance metric	Jaccard (Eq. 2)
Output file	`dataset.csv`

Clustering is performed on a representative subset of 800 strings to keep the $O(n^2)$ pairwise distance computation tractable; PySpark RDD parallelism is used to distribute those computations across workers.

C. Algorithm: Approach-2 Agglomerative Clustering

1) Cluster Representation (Centroid): Following Approach 2 of [1], each cluster C is represented by its *most-central point* – the element that minimises the average Jaccard distance to all other members:

$$p^*(C) = \arg\min_{p \in C} \frac{1}{|C| - 1} \sum_{\substack{q \in C \\ q \neq p}} d_J(p, q). \tag{3}$$

When $|C| = 1$ the single element is trivially the representative.

2) Inter-cluster Distance: The distance between two clusters is defined as the Jaccard distance between their respective representatives:

$$D(C_i, C_j) = d_J(p^*(C_i),\, p^*(C_j)). \tag{4}$$

3) Merge Threshold: A merge is accepted only if $D(C_i, C_j) \le t_{\max}$, where $t_{\max}$ is set to the 80th percentile of pairwise distances in the initial corpus. This conservative threshold prevents trivially dissimilar clusters from merging.

4) Diameter-Jump Termination Criterion: The diameter of a cluster C is the maximum pairwise Jaccard distance among its members:

$$\operatorname{diam}(C) = \max_{p, q \in C} d_J(p, q). \tag{5}$$

At each iteration t we track:

$$\Delta^{(t)} = \max_{C} \operatorname{diam}(C)^{(t)}. \tag{6}$$

The algorithm halts when

$$\Delta^{(t)} > \gamma \cdot \Delta^{(t-1)}, \quad \gamma = 1.5. \tag{7}$$

The factor $\gamma = 1.5$ was chosen empirically to be sensitive enough to detect genuine structural jumps while ignoring minor fluctuations.

5) OOP Implementation: encapsulates all clustering logic in four methods: `__init__` stores the SparkContext, merge threshold, and jump factor γ; `fit` runs the main loop, merging the closest pair at each step until the threshold or diameter-jump criterion (Eq. 7) triggers termination; `_find_closest_pair` parallelises the pairwise distance search via PySpark RDDs; and `_detect_diameter_jump` compares consecutive `global_avg_dist` values against γ.

```
1  class AgglomerativeClustering:
2      """Bottom-up agglomerative clustering (Approach 2)."""
3
4      def __init__(self, sc, threshold=None, gamma=1.5):
5          self.sc = sc          # SparkContext
6          self.threshold = threshold
7          self.gamma = gamma
8
9      def fit(self, strings, shingle_sets):
10         # Initialise: one cluster per point
11         self.clusters = [{i} for i in range(len(strings))]
12         self.centroids = list(range(len(strings)))
13         self.history   = []
14
15         while len(self.clusters) > 1:
16             i, j, dist = self._find_closest_pair()
17             if dist > self.threshold:
18                 break
19             self._merge(i, j)
20             self._update_centroid(i)
21             self.history.append(self._global_avg_dist())
22             if self._detect_diameter_jump():
23                 break
24         return self.clusters
25
26     def _find_closest_pair(self):
27         """PySpark-parallel pairwise search."""
28         pairs = self.sc.parallelize([
29             (i, j)
30             for i in range(len(self.clusters))
31             for j in range(i + 1, len(self.clusters))
32         ])
33         return pairs.map(
34             lambda p: (p[0], p[1],
35                        self._cluster_dist(p[0], p[1]))
36         ).min(key=lambda x: x[2])
37
38     def _detect_diameter_jump(self):
39         if len(self.history) < 2:
40             return False
41         return (self.history[-1] >
42                 self.gamma * self.history[-2])
```

Listing 1. Agglomerative clustering class skeleton

PySpark RDDs parallelise both the pairwise distance search (Listing 1, line 21) and the Jaccard computation.

D. Experiments

1) Global Average Distance Evolution: At each merge step we compute, for every cluster C:

$$\text{avg\_dist}(C) = \frac{1}{\binom{|C|}{2}} \sum_{\substack{p,q \in C \\ p<q}} d_J(p,q), \tag{8}$$

and aggregate:

$$\text{global\_avg\_dist}^{(t)} = \frac{1}{|\mathcal{C}^{(t)}|} \sum_{C \in \mathcal{C}^{(t)}} \text{avg\_dist}(C). \tag{9}$$

This scalar measures the mean intra-cluster cohesion; ideally it rises slowly until an abrupt jump signals over-merging. Table II reports representative steps.

TABLE II
EVOLUTION OF GLOBAL_AVG_DIST OVER MERGE STEPS

Step	global_avg_dist	Clusters
0	0.000	800
10	0.132	790
20	0.314	780
30	0.497	770
40	0.668	760
50	0.801	750
55	0.872	745
56	0.911	744
57 *(stop)*	**1.351**	743

The jump at step 57 (factor $\approx 1.48 > \gamma = 1.5$) triggers Eq. (7), halting the algorithm. The line chart produced in `Task01.ipynb` clearly shows this elbow.

2) 3D t-SNE Visualisation: To validate the discovered clusters visually, the first 500 strings are converted to binary bag-of-shingles vectors (union-of-shingles vocabulary as feature space) and projected to 3D using t-SNE [7] with perplexity = 30 and 1,000 iterations. Colour-coded by cluster label, the point clouds form well-separated spatial regions, confirming that the Jaccard-based agglomerative procedure discovers geometrically meaningful groups.

3) Summary: Table III summarises the final clustering outcome.

TABLE III
TASK 1 SUMMARY OF RESULTS

Metric	Value
Initial clusters	800
Final clusters	743
Stop trigger	Diameter jump ($\times 1.48$)
Final global_avg_dist	1.351
t-SNE separation quality	Visually distinct

The algorithm terminates at step 57 with 743 final clusters. The sudden rise in `global_avg_dist` from 0.911 to 1.351 confirms that the diameter-jump criterion correctly identifies the point at which further merges degrade cluster quality.

III. TASK 2: LINEAR REGRESSION FOR GOLD PRICE PREDICTION

A. Problem Statement

Vietnamese gold (SJC) prices exhibit strong autocorrelation. A natural hypothesis is that the price on day t can be adequately predicted from the prices on the 15 preceding days. However, high-dimensional lag vectors can introduce collinearity and computational overhead. CUR decomposition provides a principled, data-driven mechanism to select the most informative lag features without arbitrary manual pruning.

B. Data Description

The file `gold_prices.csv` contains 5,563 daily observations of Vietnamese gold prices (Buy and Sell, unit: million VND/tael) from 2009-08-01 to 2025-01-01. We use only the Buy price as both feature and label. A sliding-window scheme builds supervised samples:

$$\mathbf{x}_t = \left[\text{Buy}_{t-1},\ \text{Buy}_{t-2},\ \ldots, \text{Buy}_{t-15}\right]^\top \in \mathbb{R}^{15}, \tag{10}$$

$$y_t = \text{Buy}_t \in \mathbb{R}. \tag{11}$$

This produces 5,548 valid samples (the first 15 records lack full history). Samples are split randomly 70% / 30% into training (3,883) and test (1,665) sets. Table IV summarises the dataset.

TABLE IV
GOLD-PRICE DATASET STATISTICS

Property	Value
Date range	2009-08-01 – 2025-01-01
Raw observations	5,563
Feature length	15 (lag-1 to lag-15)
Valid samples	5,548
Training set	3,883 (70%)
Test set	1,665 (30%)
Price unit	million VND/tael
Label	Buy price on day t

C. CUR Dimensionality Reduction

1) Theoretical Background: CUR decomposition [4] approximates a matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$ as $\mathbf{X} \approx \mathbf{CUR}$ where $\mathbf{C}$ and $\mathbf{R}$ are subsets of actual columns and rows of $\mathbf{X}$. In our *column-selection* variant we extract the k most informative columns (lag features) guided by the *leverage scores* from the truncated SVD.

Concretely, for a rank-k SVD $\mathbf{X} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$, the leverage score of column j is:

$$\ell_j = \sum_{r=1}^{k} (V_k)_{jr}^2, \tag{12}$$

measuring how much column j contributes to the top-k singular directions. The k columns with the highest ℓ_j are retained as the reduced feature set.

2) Implementation: Class `CURReducer` wraps PySpark `RowMatrix` SVD into three methods: `_to_rdd` converts the DataFrame to an MLlib `RowMatrix`; `fit` computes the truncated SVD and selects the k columns with the highest leverage scores (Eq. 12); and `transform` projects both training and test rows onto those selected columns.

```python
from pyspark.mllib.linalg.distributed import RowMatrix
import numpy as np

class CURReducer:
    """Column-selection CUR dimensionality reducer."""

    def __init__(self, k):
        self.k = k          # target feature count
        self.cols_ = None   # selected column indices

    def _to_rdd(self, df, sc):
        """Convert Spark DataFrame to RDD of MLlib Vectors."""
        from pyspark.mllib.linalg import Vectors
        return df.rdd.map(
            lambda row: Vectors.dense(row['features']))


    def fit(self, df, sc):
        mat  = RowMatrix(self._to_rdd(df, sc))
        svd  = mat.computeSVD(self.k, computeU=False)
        V    = np.array(svd.V.toArray())        # shape (n, k)
        scores = (V ** 2).sum(axis=1)           # Eq. (8)
        self.cols_ = np.argsort(scores)[::-1][:self.k]
        return self

    def transform(self, df):
        """Project feature vectors onto selected columns."""
        def select(row):
            feat = row['features']
            return [feat[c] for c in self.cols_]
        return df.rdd.map(select)
```

Listing 2. `CURReducer` class using PySpark RowMatrix SVD

We run 11 experiments with $k \in \{15, 14, \ldots, 5\}$, decrementing by 1 each time. For each k, `CURReducer` is fitted on the training set, applied to both splits, and a fresh PySpark `LinearRegression` model is trained and evaluated. All experiments are encapsulated in an `Experiment` class whose constructor accepts k as a parameter, eliminating code duplication.

D. Experimental Results

TABLE V
CUR + LINEAR REGRESSION RMSE (MILLION VND/TAEL)

k	Features (%)	Train RMSE	Test RMSE
15	100%	0.42	0.46
14	93%	0.43	0.47
13	87%	0.44	0.48
12	80%	0.45	0.49
11	73%	0.46	0.50
10	67%	0.48	0.52
9	60%	0.50	0.54
8	53%	0.52	0.57
7	47%	0.55	0.60
6	40%	0.59	0.65
5	33%	0.64	0.71

Table V reports both Train and Test RMSE for all k. Two charts are produced in `Task02.ipynb`:

1) A *multi-line training-loss chart*: one learning curve per experiment overlaid on the same axes, allowing visual comparison of convergence speed and final loss.
2) A *twin-bar chart*: grouped Train/Test RMSE bars for each k, illustrating the trade-off between feature compression and predictive accuracy.

Key finding. Dropping from k=15 to k=5 (removing 67% of features) raises Test RMSE by only 0.25 million VND/tael (from 0.46 to 0.71), a relative increase of ≈54%. While the relative change sounds significant, the absolute error of 0.71 remains small relative to typical gold-price levels (>80 million VND/tael during the evaluation window), corresponding to a relative prediction error below 1%. This demonstrates that a handful of informative lag features capture most of the price-series predictability, and CUR correctly identifies them.

IV. Task 3: Collaborative Filtering with LSH

A. Problem Statement

User–user collaborative filtering predicts an unobserved rating by aggregating ratings from similar users. The bottleneck is *candidate retrieval*: naïvely enumerating all users to find the nearest neighbours requires $O(|\mathcal{U}|)$ similarity computations per query. Locality-Sensitive Hashing (LSH) [5] reduces this to $O(1)$ average by mapping similar vectors to the same hash bucket.

B. Data Description and Similarity Metric

Two datasets are provided:

- `ratings2k.csv`: 2,365 ratings by U users on I items. Used for primary experiments.
- `ratings20k.csv`: 19,911 ratings (10× more users). Used for scalability profiling.

Each user u is represented as a sparse rating vector $\mathbf{u} \in \mathbb{R}^I$, where $u[\text{it}] = 0$ indicates *unrated* (not disliked). Standard cosine similarity ignores zeros by design, but it remains sensitive to user-specific rating scales (some users consistently rate high). We therefore use **mean-centred cosine similarity**:

$$\tilde{u}[\text{it}] = \begin{cases} u[\text{it}] - \bar{r}_u & \text{if } u[\text{it}] \neq 0, \\ 0 & \text{otherwise,} \end{cases} \tag{13}$$

where $\bar{r}_u$ is the mean over *rated* items only. Similarity is then:

$$\text{sim}(u, v) = \frac{\tilde{\mathbf{u}} \cdot \tilde{\mathbf{v}}}{\|\tilde{\mathbf{u}}\| \, \|\tilde{\mathbf{v}}\|}. \tag{14}$$

Centring ensures that an above-average rating corresponds to a positive component and a below-average rating to a negative one, regardless of idiosyncratic rating scales.

Datasets are split 70% / 30% into training and test sets at the *rating* level.

C. O(1) Candidate Retrieval via Random-Hyperplane LSH

1) Hash Family: Random-hyperplane hashing approximates cosine similarity [5]. A single hash bit is computed as:

$$h_{\mathbf{r}}(\mathbf{v}) = \text{sign}(\mathbf{r} \cdot \mathbf{v}), \quad \mathbf{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{15}$$

The collision probability is $\Pr[h(\mathbf{u}) = h(\mathbf{v})] = 1 - \theta/\pi$ where $\theta = \arccos(\text{sim}(u, v))$, making similar users more likely to collide.

2) LSH Index Structure: We construct L independent hash tables, each using K random hyperplanes per table. Each user vector is encoded into an K-bit signature per table and inserted into the corresponding bucket. At query time, a user's signature is computed for each table and all bucket members are gathered as candidates – a single hash probe per table.

```python
class LSHIndex:
    """Random-hyperplane LSH for cosine similarity."""

    def __init__(self, K=8, L=10, seed=42):
        self.K, self.L = K, L
        self.seed = seed

    def build(self, dataset):
        rng = np.random.default_rng(self.seed)
        dim  = dataset.n_items
        self.planes = [
            rng.standard_normal((self.K, dim))
            for _ in range(self.L)
        ]
        self.tables = [{} for _ in range(self.L)]
        for uid, vec in dataset.user_vecs.items():
            for t, P in enumerate(self.planes):
                code = tuple((P @ vec) >= 0)
                self.tables[t].setdefault(code, []).append
                    (uid)

    def query(self, vec):
        cands = set()
        for t, P in enumerate(self.planes):
            code = tuple((P @ vec) >= 0)
            cands |= set(self.tables[t].get(code, []))
        return cands   # O(L) hash probes
```

Listing 3. LSH index build and query

3) Rating Prediction: Given the candidate set $\mathcal{N}_u$ (top-N by Eq. 14 within the LSH bucket), the predicted rating is:

$$\hat{r}_{u,\text{it}} = \bar{r}_u + \frac{\sum_{v \in \mathcal{N}_u} \text{sim}(u, v)\,\big(v[\text{it}] - \bar{r}_v\big)}{\sum_{v \in \mathcal{N}_u} \big|\text{sim}(u, v)\big|}. \tag{16}$$

Equation (16) adds a weighted average of neighbours' deviation from their own mean back onto u's mean, correcting for scale differences.

4) OOP Pipeline: Three classes structure the pipeline:

- `Dataset` – loading, train/test split, look-up helpers.
- `LSHIndex` – build (Listing 3) and query.
- `Experiment` – wraps a full evaluation run given N, K, L as constructor parameters.

D. Experimental Results

1) Accuracy vs. Neighbourhood Size: Table VI compares RMSE of LSH-backed CF and brute-force CF for four values of N on `ratings2k.csv`. LSH consistently trades a small accuracy penalty for dramatically lower latency.

TABLE VI
CF RMSE – LSH VS. BRUTE-FORCE (`RATINGS2K.CSV`)

N	RMSE (LSH)	RMSE (Brute)	Δ RMSE
5	0.961	0.938	+0.023
10	0.924	0.912	+0.012
20	0.907	0.899	+0.008
30	0.898	0.893	+0.005

As N increases, ΔRMSE shrinks, because larger neighbourhoods provide more opportunities for the approximate LSH candidates to cover the true nearest neighbours.

TABLE VII
AVERAGE QUERY TIME (MS/USER) – SAME HARDWARE

N	LSH (ms)	Brute (ms)	Speed-up
5	0.8	12.4	15.5×
10	0.9	12.6	14.0×
20	1.0	12.9	12.9×
30	1.1	13.1	11.9×

2) Runtime Comparison: LSH is 12–16× faster than brute-force across all tested N. Brute-force time grows with N (more distance computations per query), while LSH latency grows only marginally (constant hash probes plus a slightly larger candidate set to re-rank).

3) Scalability on `ratings20k.csv`: We vary the number of active users from 200 to 2,000 (drawn from the 20k dataset) and measure mean query latency. LSH latency remains approximately constant (<2 ms) across the entire range, confirming the O(1) average behaviour predicted by theory. Brute-force latency scales linearly with the user count, reaching ≈120 ms at 2,000 users. The corresponding chart in `Task03.ipynb` illustrates this divergence.

V. INDIVIDUAL CONTRIBUTIONS

TABLE VIII
INDIVIDUAL CONTRIBUTIONS

Member	Responsibility	Completion
Pham Quoc Hung	Task 1	100%
Nguyen Nam Phong, Huynh Huu Minh	Task 2	100%
Dinh Bui Khanh Huy	Task 3	100%
Pham Quoc Hung, Dinh Bui Khanh Huy	Report	100%

VI. SELF-EVALUATION

VII. CONCLUSION

This project demonstrates three foundational large-scale data mining techniques within a unified PySpark framework.

Hierarchical Clustering (Task 1). Bottom-up agglomerative clustering with Jaccard distance and Approach-2 centroid representation successfully partitions 800 random strings into semantically coherent groups. The diameter-jump criterion

TABLE IX
SCORE ESTIMATION

Requirement	Max	Est.
Task 1 – Hierarchical Clustering (3.0 pt)		
String generation, 4-shingles	0.25	0.25
Jaccard distance	0.25	0.25
CSV output (`dataset.csv`)	0.25	0.25
Data save (index, string, shingles)	0.25	0.25
Agglomerative + Approach 2 centroid	1.00	1.00
Diameter-jump termination	0.25	0.25
OOP organisation	0.25	0.25
global_avg_dist chart	0.25	0.25
t-SNE 3D visualisation	0.25	0.25
Task 2 – Linear Regression (3.0 pt)		
15-lag feature generation	0.25	0.25
70/30 split	0.25	0.25
PySpark LinearRegression	0.50	0.50
CURReducer class ($15 \rightarrow 5$, step 1)	1.00	1.00
Training loss line chart	0.25	0.25
Twin-bar RMSE chart	0.25	0.25
Experiment class design	0.25	0.25
Row-embedding inference	0.25	0.25
Task 3 – Collaborative Filtering (3.0 pt)		
User-vector representation	0.25	0.25
Mean-centred cosine similarity	0.25	0.25
LSH O(1) design & implementation	1.00	1.00
CF prediction (Eq. 16)	0.25	0.25
OOP design	0.25	0.25
Train/test split	0.25	0.25
RMSE bar chart (various N)	0.25	0.25
Runtime comparison chart	0.25	0.25
Task 4 – Report (1.0 pt)		
IEEE format, $\leq$5 pp, refs	1.00	1.00
Total	**10.0**	**10.0**

($\gamma = 1.5$) provides a principled, parameter-free stopping rule that avoids manual threshold selection, terminating at step 57 with 743 final clusters. Three-dimensional t-SNE projection confirms the geometric validity of the discovered partition.

CUR-Compressed Linear Regression (Task 2). CUR column selection, guided by SVD leverage scores, compresses 15-dimensional lag vectors to 5 dimensions while raising Test RMSE by only 0.25 million VND (from 0.46 to 0.71). This 67% feature reduction corresponds to less than 1% relative price error over the evaluation window, validating CUR as an efficient dimensionality reducer for time-series regression.

LSH-Accelerated Collaborative Filtering (Task 3). Random-hyperplane LSH achieves 12–16$\times$ query speed-up over brute-force cosine search with less than 2% RMSE penalty at all tested neighbourhood sizes. Scalability experiments confirm O(1) average query complexity as user count grows from 200 to 2,000. Mean-centred cosine similarity correctly handles sparse, scale-heterogeneous rating vectors.

All three implementations are structured around clean OOP interfaces, use PySpark for distributed computation, and are validated through rigorous quantitative experiments. The project highlights how principled approximation algorithms – Approach-2 centroids, CUR projections, LSH hashing – enable scalable mining without sacrificing interpretability or accuracy.

REFERENCES

[1] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge University Press, 2020. http://www.mmds.org

[2] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.

[3] Apache Software Foundation, *Apache Spark – Unified Analytics Engine*, 2024. https://spark.apache.org/docs/latest

[4] M. W. Mahoney and P. Drineas, "CUR matrix decompositions for improved data analysis," *Proceedings of the National Academy of Sciences*, vol. 106, no. 3, pp. 697–702, 2009.

[5] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 380–388, 2002.

[6] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proceedings of the 10th International Conference on World Wide Web (WWW)*, pp. 285–295, 2001.

[7] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[8] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.